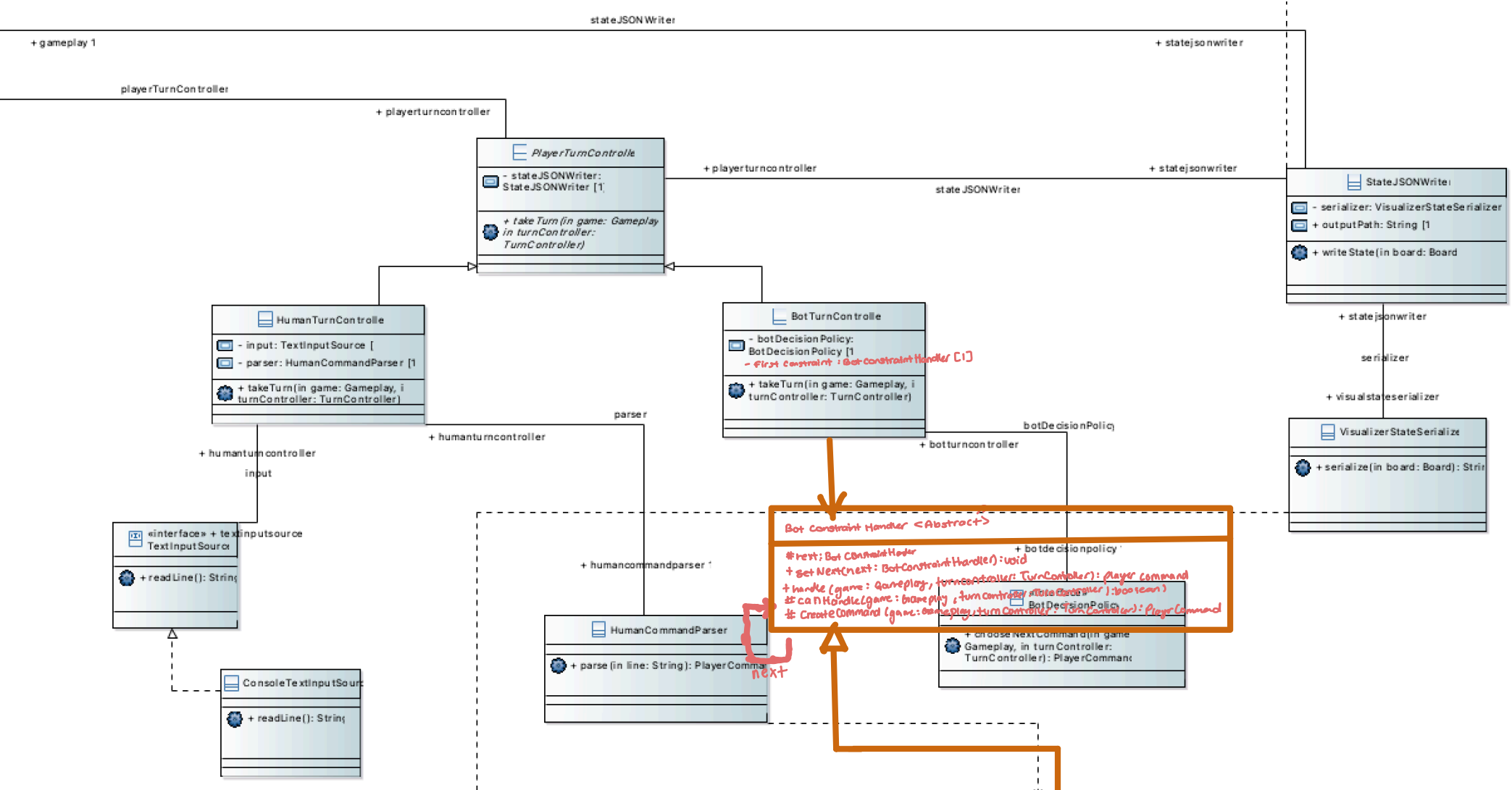
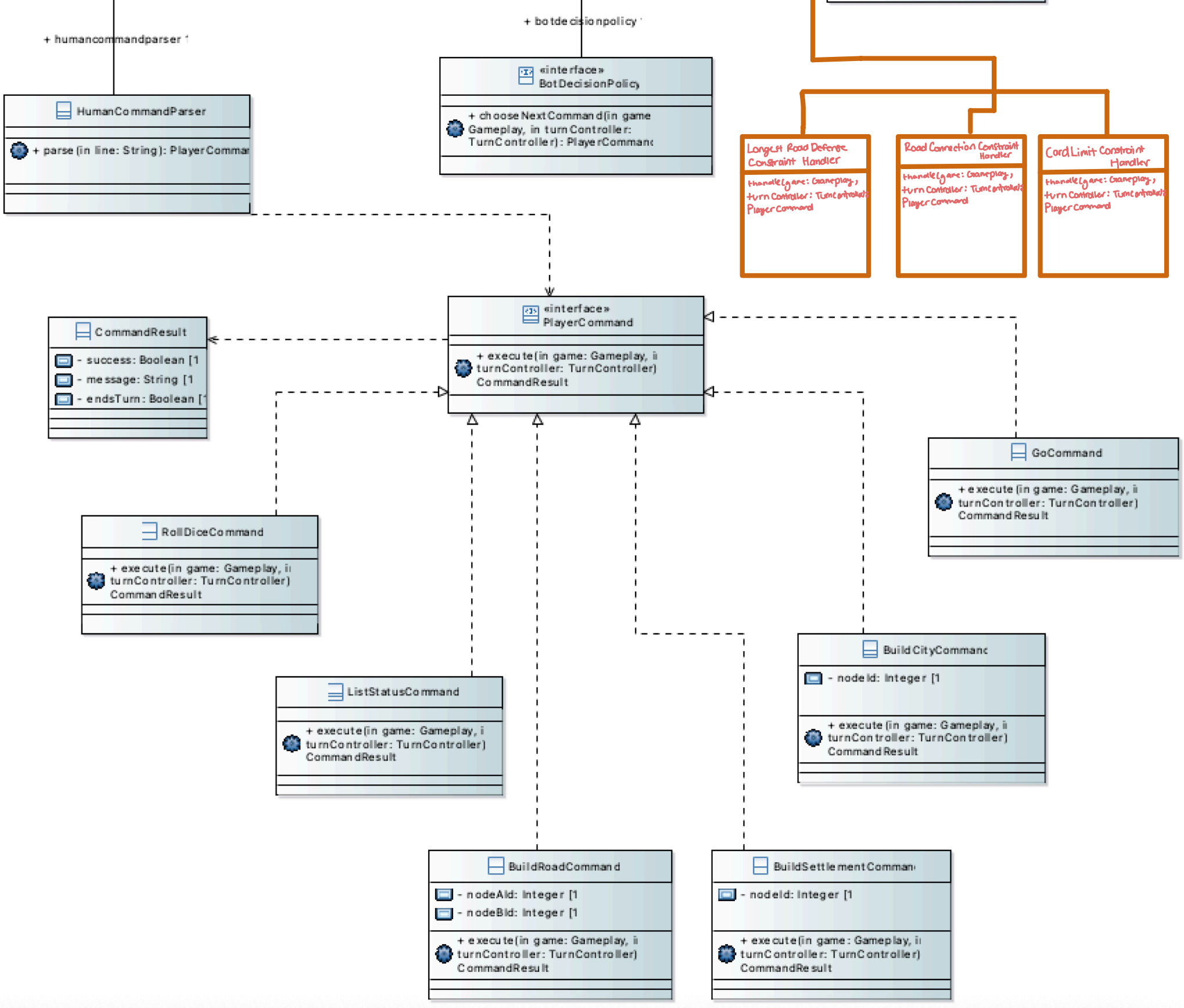

stateJSONWriter
+ gameplay 1
+ statejsonwriter
playerTurnController
+ playerturncontroller
PlayerTurnControlle
- stateJSONWriter: StateJSONWriter [1]
+ takeTurn(in game: Gameplay, in turnController: TurnController)
+ playerturncontroller
+ statejsonwriter
stateJSONWriter
StateJSONWriter
- serializer: VisualizerStateSerializer
+ outputPath: String [1
+ writeState(in board: Board
+ statejsonwriter
serializer
+ visualstateserializer
VisualizerStateSerialize
+ serialize(in board: Board): Strin
HumanTurnControlle
- input: TextInputSource [
- parser: HumanCommandParser [1
+ takeTurn(in game: Gameplay, in turnController: TurnController)
BotTurnControlle
- botDecisionPolicy: BotDecisionPolicy [1
- first constraint : BotConstraintHandler [1]
+ takeTurn(in game: Gameplay, in turnController: TurnController)
parser
+ humanturncontroller
botDecisionPolicy
+ botturncontroller
+ humanturncontroller
input
«interface» + textinputsource
TextInputSource
+ readLine(): String
Bot Constraint Handler <Abstract>
#next: BotConstraintHandler
+setNext(next: BotConstraintHandler): void
+handle(game: Gameplay, turncontroller: TurnController): PlayerCommand
#canHandle(game: Gameplay, turnController: TurnController): boolean
#CreateCommand(game: Gameplay, turnController: TurnController): PlayerCommand
+ botdecisionpolicy
+ humancommandparser
BotDecisionPolicy
+ chooseNextCommand(in game: Gameplay, in turnController: TurnController): PlayerCommand
HumanCommandParser
+ parse(in line: String): PlayerCommand
next
ConsoleTextInputSourc
+ readLine(): String


+ humancommandparser
+ botdecisionpolicy
HumanCommandParser
+ parse (in line: String): PlayerCommar
«interface»
BotDecisionPolicy
+ chooseNextCommand(in game
Gameplay, in turnController:
TurnController): PlayerCommanc
Longest Road Defense Constraint Handler
+handle(game: Gameplay,
+turnController: TurnController):
PlayerCommand
Road Connection Constraint Handler
+handle(game: Gameplay,
+turnController: TurnController):
PlayerCommand
Card Limit Constraint Handler
+handle(game: Gameplay,
+turnController: TurnController):
PlayerCommand
CommandResult
- success: Boolean [1
- message: String [1
- endsTurn: Boolean [1
«interface»
PlayerCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
GoCommand
+ execute (in game: Gameplay, i
turnController: TurnController)
CommandResult
RollDiceCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
ListStatusCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildCityCommanc
- nodeId: Integer [1
+ execute (in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildRoadCommand
- nodeAId: Integer [1
- nodeBId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildSettlementComan
- nodeId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult